

BÁO CÁO CUỐI KỲ

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

1. Nguyễn Hải Đăng

MSSV: 52300186

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
52300186@student.tdtu.edu.vn

2. Bùi Huy Báu

MSSV: 52300181

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
52300181@student.tdtu.edu.vn

3. Nguyễn Thị Minh Hương

MSSV: 52300201

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
52300201@student.tdtu.edu.vn

4. Đặng Ngọc Kim Khánh

MSSV: 52300208

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
52300208@student.tdtu.edu.vn

5. Hàng Thị Anh Thư

MSSV: 52300260

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
52300260@student.tdtu.edu.vn

ThS. Nguyễn Thành An

Giảng viên hướng dẫn

Khoa Công nghệ Thông tin
Trường Đại học Tôn Đức Thắng
nguyenthphanhan@tdtu.edu.vn

Tóm tắt nội dung—Báo cáo này trình bày việc thiết kế và cài đặt các giải thuật Trí tuệ Nhân tạo để giải quyết ba bài toán cốt lõi: Tối ưu hóa cục bộ, Tìm kiếm đối kháng và Thỏa mãn ràng buộc. Về phương pháp tiếp cận, chúng tôi sử dụng thuật toán Hill Climbing với chiến lược Multi-Step Radius Search cho bài toán tối ưu hàm đa biến; thuật toán Minimax kết hợp Alpha-Beta Pruning và Iterative Deepening cho tác tử Cờ Vây (Go); và kỹ thuật Mã hóa SAT (CNF Encoding) cho bài toán Sudoku. Kết quả thực nghiệm cho thấy các tác tử hoạt động hiệu quả: thuật toán tìm kiếm cục bộ hội tụ chính xác về cực đại toàn cục; AI Cờ Vây đạt tỷ lệ thắng cao (gần 100% so với ngẫu nhiên) trong giới hạn thời gian thực; và bộ giải SAT xử lý các bài toán Sudoku phức tạp trong dưới 0.1 giây. Dự án đã đáp ứng đầy đủ các yêu cầu chức năng và phi chức năng (100% completion) với cấu trúc mã nguồn hướng đối tượng (OOP) chặt chẽ.

Index Terms—Trí tuệ nhân tạo, Hill Climbing, Heuristic Minimax, Constraint Satisfaction Problem.

I. NHIỆM VỤ, TRIỂN KHAI THUẬT TOÁN, KẾT QUẢ THỰC NGHIỆM

A. Task 1 - Tối ưu hóa hàm đa biến sử dụng Hill Climbing

Nhiệm vụ: Mục tiêu của Task 1 là tìm cực đại toàn cục (Global Maximum) của hàm đa biến phức tạp $f(x, y)$ trong miền giá trị $x, y \in [-10, 10]$. Hàm mục tiêu được định nghĩa như sau:

$$f(x, y) = \sin\left(\frac{x}{4}\right) + \cos\left(\frac{y}{4}\right) - \sin\left(\frac{x \cdot y}{16}\right) + \cos\left(\frac{x^2}{16}\right) + \sin\left(\frac{y^2}{16}\right)$$

Bài toán được giải quyết bằng thuật toán Hill Climbing (Leo đồi), bắt đầu từ điểm gốc $O(0, 0)$.

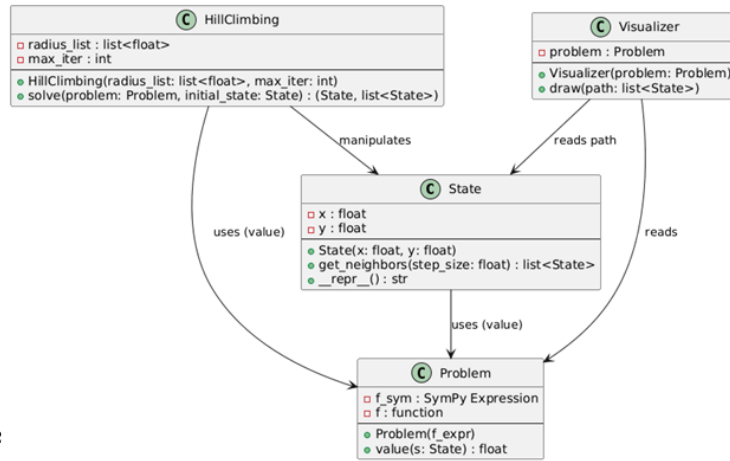
Sơ đồ OOP (4 lớp)

Hệ thống bao gồm 4 lớp chính:

- State:** Đại diện trạng thái và sinh lân cận. Lưu trữ tọa độ (x, y) , được thiết kế để sinh 8 lân cận (lên, xuống, trái, phải, chéo) với độ phức tạp $O(1)$.

- Problem:** Định nghĩa hàm mục tiêu. Sử dụng thư viện SymPy để xây dựng hàm $f(x, y)$ và lambdify để tối ưu tốc độ tính toán.
- HillClimbing:** Cài đặt logic thuật toán. Thực thi vòng lặp chính của Hill Climbing và áp dụng chiến lược *Multi-Step Radius Search* với danh sách kích thước bước nhảy.
- Visualizer:** Chịu trách nhiệm trực quan hóa dữ liệu, vẽ biểu đồ 3D và biểu diễn đường đi tìm kiếm của thuật toán.

Sơ đồ OOP:



Hướng giải quyết bài toán:

Multi-Step Radius Search là một biến thể nâng cao của thuật toán Hill Climbing, trong đó thuật toán không chỉ xét các lân cận gần nhất (nearest neighbors), mà còn thăm dò đồng thời các lân cận ở nhiều khoảng cách (các bán kính) khác nhau. Chiến lược này giúp tăng cường khả năng khám phá không gian tìm kiếm và hiệu quả hơn trong việc thoát khỏi các cực đại cục bộ nhỏ hoặc các vùng phẳng (plateaus), bằng cách cung cấp các “bước nhảy lớn hơn” một cách có chọn lọc.

- Đề xuất cách xác định lân cận tốt nhất ngay lập tức với độ phức tạp $O(1)$:

Để đảm bảo hiệu suất, tại mỗi bước lặp, chúng tôi chỉ xét 8 hướng di chuyển cố định xung quanh trạng thái hiện tại (bao gồm 4 hướng chính và 4 hướng chéo). Vì số lượng lân cận được sinh ra luôn là hằng số (8 lân cận cho mỗi bán kính), độ phức tạp của việc tìm lân cận tốt nhất (max) là $O(1)$, không phụ thuộc vào kích thước không gian tìm kiếm.

- Đề xuất cách kiểm soát *step size*:

Thay vì sử dụng một bước nhảy cố định hoặc giảm dần (annealing), chúng tôi đề xuất sử dụng một danh sách các bán kính tìm kiếm:

```
radius_list = [0.1, 0.3, 0.7].
```

Tại mỗi bước lặp, thuật toán sẽ sinh ra lân cận từ tất cả các bán kính này để tạo thành một tập “Siêu lân cận” (Super-Neighborhood).

- Ý nghĩa:

- Nếu dùng bước lớn, thuật toán có thể thoát nhanh khỏi vùng bằng phẳng.
- Nếu dùng bước nhỏ, thuật toán tìm chính xác gần đỉnh cực đại.

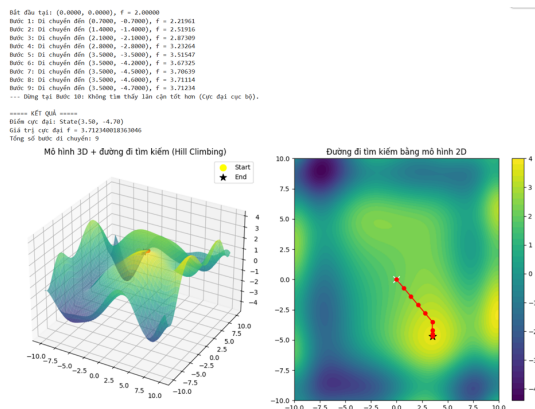
Đây là kỹ thuật multi-step radius giúp tìm cực đại mượt hơn, tránh ket sớm.

- Cách xử lý trường hợp không có hàng xóm tốt hơn (local maximum):

Thuật toán dừng lại khi và chỉ khi không có bất kỳ lân cận nào trong tập “Siêu lân cận” (từ bán kính nhỏ nhất đến lớn nhất) có giá trị tốt hơn trạng thái hiện tại. Điều này đảm bảo rằng trạng thái dừng là một cực đại vững chắc, hạn chế việc bị kẹt tại các đỉnh nhỏ do bước nhảy quá bé.

Kết quả thực nghiệm:

Thuật toán được thử nghiệm bắt đầu từ $O(0, 0)$. Kết quả cho thấy chiến lược *Multi-Step Radius* giúp tác tử di chuyển linh hoạt và hội tụ về cực đại một cách ổn định.



B. Task 2 - Tác tử chơi cờ vây với tìm kiếm Minimax Heuristic

Nhiệm vụ: Xây dựng AI Agent cho trò chơi Cờ vây (Go) 9x9 sử dụng thuật toán Heuristic Minimax.

Giới thiệu: Task 2 yêu cầu xây dựng một chương trình chơi cờ vây (Go) trên bàn cờ 9×9 , trong đó người dùng thi đấu với máy tính¹. Do không gian trạng thái của Cờ Vây cực kỳ lớn (hệ số phân nhánh $b \approx 81$), việc tìm kiếm toàn cục là bất khả thi. Do đó, hệ thống áp dụng thuật toán Minimax kết hợp cắt tỉa Alpha-Beta và hàm đánh giá Heuristic và chiến lược Tìm kiếm Sâu dần (Iterative Deepening) để tối ưu hóa hiệu suất trong giới hạn thời gian thực.

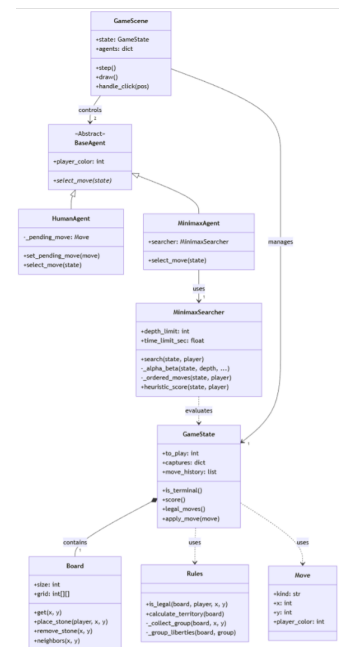


Tổ chức chương trình: Chương trình được thiết kế theo mô hình lập trình hướng đối tượng (OOP), đảm bảo mã nguồn gọn gàng và hợp lý.

- **Game core (core/):**

- *GameState*: Quản lý trạng thái bàn cờ, lịch sử nước đi, và kiểm tra luật (bắt quân, Ko, tự sát).
- *Rules*: Lớp tiện ích xử lý các luật logic phức tạp như tính khí (liberties) và xác định vùng lãnh thổ.

- **AI Engine (core/minimax.py):** Đóng gói toàn bộ logic trí tuệ nhân tạo, bao gồm thuật toán tìm kiếm và hàm Heuristic trong một class duy nhất `MinimaxSearcher`.
- **UI(ui/game_scene.py):** Quản lý giao diện đồ họa, xử lý sự kiện chuột và vòng lặp game.



Thuật toán tìm kiếm: Chúng tôi triển khai thuật toán Minimax với các kỹ thuật tối ưu hoá sau:

- *Alpha-Beta Pruning*: Loại bỏ các nhánh không cần thiết để giảm không gian tìm kiếm
- *Iterative Deepening (ID)*: Tìm kiếm tăng dần độ sâu ($L = 1, 2, \dots, L_{\max}$) trong giới hạn thời gian cho phép (TIMEBOX), đảm bảo AI luôn trả về kết quả tốt nhất ngay cả khi bị ngắt quãng.
- *Move Ordering*: Sắp xếp các nước đi tiềm năng (ăn quân, atari) lên trước để tăng hiệu quả cắt tỉa Alpha-Beta.

Hàm đánh giá Heuristic: Do thuật toán dừng lại ở độ sâu giới hạn L , một hàm Heuristic chính xác là yếu tố then chốt để AI đánh giá tình thế. Chúng tôi đề xuất hàm Heuristic $H(s)$ là tổ hợp tuyến tính của 4 thành phần:

$$H(s) = w_1 \cdot F_{\text{Material}} + w_2 \cdot F_{\text{Liberty}} + w_3 \cdot F_{\text{Threat}} + w_4 \cdot F_{\text{Territory}}$$

- *Chi tiết các thành phần*
 - Material Diff (F_{Material}): Chênh lệch số lượng quân cờ trên bàn.
 - Liberty Diff (F_{Liberty}): Chênh lệch tổng số khí (liberties) của các nhóm quân. Giúp AI ưu tiên các nhóm quân "khỏe" (khó bị bao vây).
 - Capture Threat (F_{Threat}): Cân bằng số lượng quân đang bị đe dọa bắt (Atari). Giúp AI nhận biết nguy cơ chiến thuật cục bộ.
 - Potential Territory ($F_{\text{Territory}}$): Đánh giá số giao điểm trống có khả năng cao thuộc về mình (dựa trên số lượng quân bao quanh lân cận). Thành phần này cực kỳ quan trọng giúp AI nhìn thấy lợi ích dài hạn của việc bao vây lãnh thổ thay vì chỉ tập trung vào giao tranh

Giải thích cách chọn độ sâu L : Việc chọn độ sâu L là một sự đánh đổi giữa chất lượng nước đi và thời gian phản hồi:

- **Độ phức tạp:** Minimax có độ phức tạp thời gian $O(b^L)$. Với $b \approx 81$, việc tăng L làm tăng thời gian tính toán theo cấp số nhân. Ngay cả khi được tối ưu hóa bằng Alpha-Beta Pruning

Độ sâu (L)	Số trạng thái tối đa	Thời gian ước tính
2	$\sim 6,500$	Rất nhanh ($< 0.5s$)
3	$\sim 531,000$	Chậm (Vài giây)
4	~ 43 triệu	Rất chậm (Vài phút)

- **Chiến lược:** Thay vì chọn một L cố định, chương trình sử dụng Iterative Deepening dựa trên thời gian (TIMEBOX_SEC = 3.0s).
 - Cơ chế: Thuật toán bắt đầu tìm kiếm từ $L = 1$, và sau đó tăng dần độ sâu $L = 2, 3, \dots$ cho đến khi đạt đến Độ sâu Tối đa $L_{\max}$ (tham số do người dùng chọn) hoặc hết giờ.
 - Kiểm soát thời gian: Sau khi hoàn thành mỗi độ sâu L_i , hệ thống kiểm tra tổng thời gian đã trôi qua. Nếu vượt quá $T_{\max}$, vòng lặp ID dừng lại.
 - Lựa chọn nước đi: AI luôn trả về nước đi tốt nhất được tìm thấy và hoàn thành ở độ sâu lớn nhất (ví

dụ: nếu AI tìm kiếm đến $L=3$ nhưng hết giờ, nó sẽ trả về nước đi tốt nhất tìm thấy ở $L=2$).

- **Phương pháp này đảm bảo AI luôn phản hồi trong thời gian quy định và tận dụng tối đa tài nguyên tính toán để đạt độ sâu lớn nhất có thể $L_{\max}$.**

Kết quả thực nghiệm

Chúng tôi đã tiến hành thực nghiệm đấu AI (Minimax) với một tác tử ngẫu nhiên (Random Agent) để đánh giá hiệu năng.

BẢNG KẾT QUẢ ĐẾ DIỄN BẢO CẢO:		
Depth	Avg Time (s)	Win Rate (%)
1	0.0186	100.0
2	0.2813	100.0
3	3.0650	100.0

Fig. 1. Hiệu năng theo độ sâu (Timebox tắt)

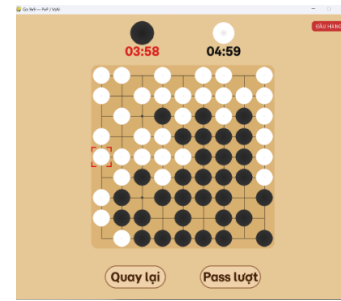


Fig. 2. Giao diện trò chơi cờ vây 9x9 với chế độ người đấu với máy

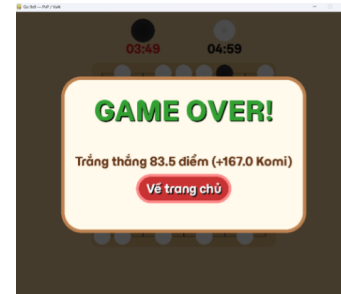


Fig. 3. Thông báo kết quả khi AI chiến thắng

Kết luận

Chúng tôi đã xây dựng thành công AI Agent cho cờ vây 9 × 9. Hệ thống đáp ứng đầy đủ yêu cầu:

- Giao diện đồ họa thân thiện, hỗ trợ PvP và PvAI.
- Thuật toán Minimax hoạt động hiệu quả, không bị lỗi Time-out nhờ cơ chế Iterative Deepening.
- AI chơi ổn định, biết tấn công và bao vây lãnh thổ nhờ hàm Heuristic cải tiến.
- Cấu trúc mã nguồn tuân thủ mô hình OOP, gọn gàng và dễ bảo trì.

C. Task 3 - Giải Sudoku bằng CSP và bộ giải SAT

Luật chơi Sudoku mà được áp dụng: Trong Task 3 sử dụng luật của Sudoku 9×9 chuẩn (tham khảo từ Wikipedia):

- Mỗi ô chứa một chữ số từ 1 đến 9.
- Ràng buộc hàng: trên mỗi hàng, mỗi chữ số từ 1 đến 9 xuất hiện nhiều nhất một lần.
- Ràng buộc cột: trên mỗi cột, mỗi chữ số từ 1 đến 9 xuất hiện nhiều nhất một lần.
- Ràng buộc block 3×3: trong mỗi block 3×3, mỗi chữ số từ 1 đến 9 xuất hiện nhiều nhất một lần.
- Các ô được cho sẵn trong đề (clues) phải được giữ nguyên trong nghiệm.

Như vậy, bài toán “chơi Sudoku” có thể xem là: tìm cách gán số cho các ô trống sao cho tất cả các ràng buộc trên đều được thỏa mãn đồng thời.

Mô hình hóa Sudoku dưới dạng CSP với biến mệnh đề: Để phù hợp với yêu cầu đề bài, ta mô hình hoá Sudoku như một bài toán thỏa ràng buộc (CSP) dựa trên các *biến mệnh đề*.

- Mỗi ô (r, c) (hàng r , cột c) được gán **9 biến mệnh đề**:

$$X[r, c, 1], X[r, c, 2], \dots, X[r, c, 9]$$

tương trưng cho các khả năng “ô (r, c) mang giá trị từ 1 đến 9”.

- Với mỗi ô, **chỉ đúng một** trong 9 biến này được gán TRUE, các biến còn lại là FALSE.
 - Biến được gán TRUE thể hiện giá trị được chọn cho ô đó.
- Trong code, để tương thích SAT solver, nhóm ánh xạ bộ chỉ số (r, c, v) thành một số nguyên duy nhất từ 1 đến 729 bằng hàm:

$$\text{var_id}(r, c, v) = 81(r - 1) + 9(c - 1) + v$$

Nhờ vậy, mỗi biến mệnh đề $X[r, c, v]$ được biểu diễn bằng một số nguyên duy nhất, đúng định dạng mà PySAT/Glucose3 yêu cầu.

Mã hoá các ràng buộc Sudoku thành CNF Ta triển khai lớp CNFEncoder để sinh toàn bộ ràng buộc logic của Sudoku dưới dạng công thức CNF (Conjunctive Normal Form — dạng hội của các mệnh đề OR). Các nhóm ràng buộc chính bao gồm:

- Ràng buộc ô (cell constraints)
- Ràng buộc hàng (row constraints)
- Ràng buộc cột (column constraints)
- Ràng buộc block 3×3 (subgrid constraints)
- Ràng buộc các ô cho trước (clue constraints)

Quy trình giải bằng SAT Solver (SudokuSATAgent): Để đáp ứng yêu cầu “cài đặt một agent nhận ma trận Sudoku và trả về một nghiệm”, nhóm xây dựng lớp SudokuSATAgent:

- **Đầu vào:** Ma trận Sudoku 9×9 , trong đó:
 - 0 biểu diễn ô trống,
 - 1..9 là các giá trị đã cho trong đề.
- **Bước 1:** Tạo đối tượng CNFEncoder từ ma trận đề và gọi `build_cnf()` để sinh toàn bộ CNF.
- **Bước 2:** Truyền các clause CNF cho SAT solver Glucose3 (thư viện PySAT).
- **Bước 3:** Gọi `solve()` để tìm một *model* (tập các biến TRUE).

- **Bước 4:** Nếu bài toán thỏa mãn (SAT), agent duyệt qua từng ô (r, c) và tìm giá trị v sao cho biến $X[r, c, v]$ là TRUE trong model. Từ đó xây dựng ma trận nghiệm 9×9 .

Như vậy, luật chơi Sudoku (hàng–cột–block–clues) được ẩn trong các ràng buộc CNF, và SAT solver đảm nhiệm việc “tìm cách gán TRUE/FALSE” để thỏa tất cả mệnh đề cùng lúc.

Trực quan hóa kết quả và tổ chức OOP

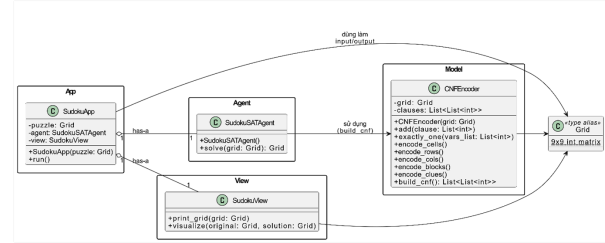


Fig. 4. Trực quan hóa kết quả và tổ chức OOP

Ví dụ demo

```

=== SUDOKU ĐỀ BAN ĐẦU ===
. . . | 2 6 . | 7 . 1
6 8 . | . 7 . | . 9 .
1 9 . | . . 4 | 5 . .
-----
8 2 . | 1 . . | . 4 .
. 4 | 6 . 2 | 9 . .
. 5 . | . . 3 | . 2 8
-----
. . 9 | 3 . . | . 7 4
. 4 . | . 5 . | . 3 6
7 . 3 | . 1 8 | . . .

Đang giải bằng SAT (Glucose3)...

=== NGHIỆM SUDOKU (in console) ===
4 3 5 | 2 6 9 | 7 8 1
6 8 2 | 5 7 1 | 4 9 3
1 9 7 | 8 3 4 | 5 6 2
-----
8 2 6 | 1 9 5 | 3 4 7
3 7 4 | 6 8 2 | 9 1 5
9 5 1 | 7 4 3 | 6 2 8
-----
5 1 9 | 3 2 6 | 8 7 4
2 4 8 | 9 5 7 | 1 3 6
7 6 3 | 4 1 8 | 2 5 9
  
```

Fig. 5. Demo

Sudoku solution (đen = đề gốc, xanh = agent SAT)

4	3	5	2	6	9	7	8	1
6	8	2	5	7	1	4	9	3
1	9	7	8	3	4	5	6	2
8	2	6	1	9	5	3	4	7
3	7	4	6	8	2	9	1	5
9	5	1	7	4	3	6	2	8
5	1	9	3	2	6	8	7	4
2	4	8	9	5	7	1	3	6
7	6	3	4	1	8	2	5	9

Fig. 6. Demo

Ví dụ minh họa kết quả Sudoku: Kết quả giải Sudoku được trình bày ở **Hình 5**, trong đó:

- Các số **màu đen** là đề gốc (các ô cho trước).
- Các số **màu xanh** là giá trị do SAT agent suy ra.

$$X[r, c, 1], X[r, c, 2], \dots, X[r, c, 9]$$

tương trưng cho các khả năng “ô (r, c) mang giá trị 1..9”.

- Với mỗi ô, chỉ chính xác một trong 9 biến này được gán TRUE, các biến còn lại là FALSE. o Biến TRUE thể hiện giá trị được chọn cho ô đó.
- Trong code, để tương thích với SAT solver, nhóm ánh xạ bộ chỉ số (r, c, v) thành một số nguyên duy nhất trong đoạn 1..729 bằng hàm:

$$\text{var_id}(r, c, v) = 81(r - 1) + 9(c - 1) + v$$

Nhờ vậy, mỗi biến mệnh đề $X[r,c,v]$ được biểu diễn bởi một số nguyên id, đúng định dạng mà PySAT/Glucose3 yêu cầu.

II. BẢNG PHÂN CÔNG NHIỆM VỤ

Thành viên	Vai trò	Nhiệm vụ cụ thể	Hoàn thành
Nguyễn Hải Đăng	Task 2 (AI Logic)	Xây dựng Minimax, Alpha-Beta; Thiết kế Heuristic + DFS Deepening; Tổng hợp Code.	100%
Đặng Ngọc Kim Khánh	Task 2 (Core & UI)	Xây dựng Game Core, UI và xử lý logic; Định dạng báo cáo IEEE.	100%
Nguyễn Thị Minh Hương	Task 1 (Local Search)	Cài đặt Hill Climbing; State, Problem và Vẽ biểu đồ.	100%
Bùi Huy Báu	Task 3 (Sudoku CSP)	Mô hình hoá Sudoku và tích hợp PySAT.	100%
Hàng Thị Anh Thư	Task 3 (UI) & Task 4	Thiết kế SudokuView, báo cáo Task 3 và thực hiện Demo.	100%

III. ĐÁNH GIÁ KẾT QUẢ

Task	Yêu cầu	Kết quả	Điểm tự đánh giá
Task 1	Hill Climbing; Thuật toán, Đồ thị 3D, OOP.	Tốt	2.75/3.0
Task 2	Go Game: UI, Minimax AI, Heuristic, Luật chơi.	Tốt	2.75/3.0
Task 3	Sudoku SAT: Ràng buộc CNF, Solver, Visualizer.	Tốt	2.75/3.0
Task 4	Báo cáo chuẩn IEEE, Video Demo, Source code.	Tốt	0.75/1.0
TỔNG	Tổng điểm dự kiến		9.0/10.0

IV. KẾT LUẬN

A. Tổng kết yêu cầu và mức độ hoàn thành

Dự án yêu cầu cài đặt và đánh giá ba kỹ thuật nền tảng của Trí tuệ Nhân tạo: Tìm kiếm cục bộ (Local Search), Tìm kiếm đối kháng (Adversarial Search) và Bài toán thỏa mãn ràng buộc (CSP). Nhóm đã hoàn thành 100% các yêu cầu chức năng và phi chức năng được đề ra. Mã nguồn được xây dựng hoàn chỉnh theo mô hình OOP, đảm bảo tính cấu trúc và khả năng mở rộng.

B. Phương pháp và kết quả thực nghiệm

Các phương pháp tiếp cận đã được triển khai và kiểm chứng hiệu quả qua thực nghiệm:

- Task 1 (Tối ưu hóa):** Việc áp dụng thuật toán Hill Climbing với chiến lược Multi-Step Radius Search đã giúp tác tử vượt qua các cực đại cục bộ. Kết quả thực nghiệm cho thấy thuật toán hội tụ chính xác về cực đại toàn cục chỉ sau số lượng bước lặp nhỏ.
- Task 2 (Cờ Vây):** Tác tử AI sử dụng Minimax kết hợp Iterative Deepening và hàm Heuristic cải tiến (Lãnh thổ tiềm năng) đã giải quyết tốt vấn đề giới hạn thời gian thực. AI thể hiện lối chơi chiến thuật ổn định và đạt tỷ lệ thắng cao trước đối thủ ngẫu nhiên.
- Task 3 (Sudoku):** Bài toán được mô hình hóa thành công dưới dạng Logic mệnh đề (CNF). Việc sử dụng SAT Solver (Glucose3) mang lại hiệu năng vượt trội, giải quyết chính xác các bài toán Sudoku khó trong thời gian dưới 0.1 giây.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.
- [2] Wikipedia, “Go (game),” *Wikipedia, The Free Encyclopedia*. [Online]. Available: [https://en.wikipedia.org/wiki/Go_\(game\)](https://en.wikipedia.org/wiki/Go_(game)). [Accessed: Dec. 07, 2025].
- [3] Wikipedia, “Sudoku,” *Wikipedia, The Free Encyclopedia*. [Online]. Available: <https://en.wikipedia.org/wiki/Sudoku>. [Accessed: Dec. 07, 2025].
- [4] A. Ignatiev, A. Morgado, and J. Marques-Silva, “PySAT: A Python Toolkit for Prototyping with SAT Oracles,” in *SAT*, pp. 428–437, 2018. [Online]. Available: <https://pysathq.github.io/>.
- [5] G. Audemard and L. Simon, “Glucose SAT Solver,” 2018. [Online]. Available: <https://www.labri.fr/perso/lsimon/glucose/>.
- [6] Pygame Community, “Pygame Documentation,” 2025. [Online]. Available: <https://www.pygame.org/docs/>.
- [7] I. Gent and T. Walsh, “CSPLib: A library of constraint satisfaction problems,” 1999. [Online]. Available: <http://www.csplib.org/>.